

在 OpenCode 中实现自动化代码审查 workflow

概述

本文档介绍如何将” Kimi 生成代码 -> GPT-5.4 审查 -> Kimi 修改代码” 的手动 workflow，在 OpenCode 环境中实现完全自动化。

核心思路

通过 OpenCode 的**多智能体协作**和**专业技能 (Skills)** 功能，构建一个自动化的开发-审查-修补循环，实现以下流程： 1. 一个智能体使用 Kimi 生成代码 2. 自动触发 GPT-5.4 进行代码审查 3. 根据审查建议自动修改代码

实现步骤

第一步：配置多模型路由

在 OpenCode 的配置文件为不同智能体指定模型，确保能调用正确的 AI。

配置文件路径： ~/.config/opencode/config.json 或项目根目录的 ./opencode.json

```
{
  "$schema": "https://opencode.ai/config.json",
  "provider": {
    "anthropic": {
      "options": { "apiKey": "{env:ANTHROPIC_API_KEY}" },
      "models": { "claude-sonnet-4.6": { "name": "Claude Sonnet 4.6" } }
    },
    "openai": {
      "options": { "apiKey": "{env:OPENAI_API_KEY}" },
      "models": { "gpt-5.4": { "name": "GPT-5.4" } }
    }
  }
}
```

第二步：定义自动化 workflow

使用 OpenCode 的智能体系统自动拆解任务并分配智能体。

操作指令：

`omoc ultrawork` "为当前项目实现一个新功能：[在此处描述你的具体功能需求]"

自动化流程： 1. **任务拆解：** 将需求拆解为开发、测试、代码审查等子任务 2. **智能体分配：** 自动调度 backend-engineer 智能体负责开发 3. **模型调用：** 使用配置的模型（如 Kimi K2.5）编写代码

第三步：集成审查技能

通过配置 Superpowers 技能实现自动化的代码审查。

配置文件路径： ~/.config/oh-my-opencode/config.yaml

```
default_skills:
  - superpowers/tdd
  - superpowers/code-review # 确保包含代码审查技能
```

自动化流程详解

workflow 执行过程

1. 开发阶段

- 用户发出 `omoc ultrawork` 指令
- OpenCode 调度开发智能体
- 智能体使用配置的模型（如 Kimi）编写代码

2. 自动审查阶段

- 开发完成后自动触发代码审查
- 调用 GPT-5.4 模型进行第二意见审查
- 生成详细的代码改进建议文档

3. 自动修补阶段

- 审查意见反馈给开发智能体
- 智能体根据建议自动修改代码
- 完成代码优化无需人工介入

进阶功能：手动触发审查

在需要时可手动触发 GPT-5.4 审查：

`use skill tool to load superpowers/code-review`；对当前项目进行全面代码审查，重点关注架构设计和潜在Bug

优势对比

对比项	手动流程	OpenCode 自动化
工具切换	需要切换 VSC 和 OpenCode	单一环境完成
人工干预	需要多次手动操作	完全自动化
效率	较低	高效
一致性	依赖人工记忆	配置化保证一致性

实施建议

1. 环境准备

- 确保 OpenCode 版本支持多智能体功能
- 配置好所有需要的 API 密钥
- 安装必要的插件和技能

2. 流程优化

- 根据项目特点调整智能体配置
- 定期更新技能库
- 监控自动化流程的执行效果

3. 质量保障

- 设置代码质量检查点
- 定期人工抽检自动化结果
- 建立问题反馈机制

总结

通过 OpenCode 的多智能体协作和技能扩展功能，可以将原本需要跨工具、手动操作的开发-审查-修补流程，完全整合到单一环境中实现自动化。这不仅提高了开发效率，还保证了代码质量的一致性。

最终效果：只需一个指令即可启动完整的自动化代码开发和审查流程，让 AI 助手真正成为高效的开发伙伴。